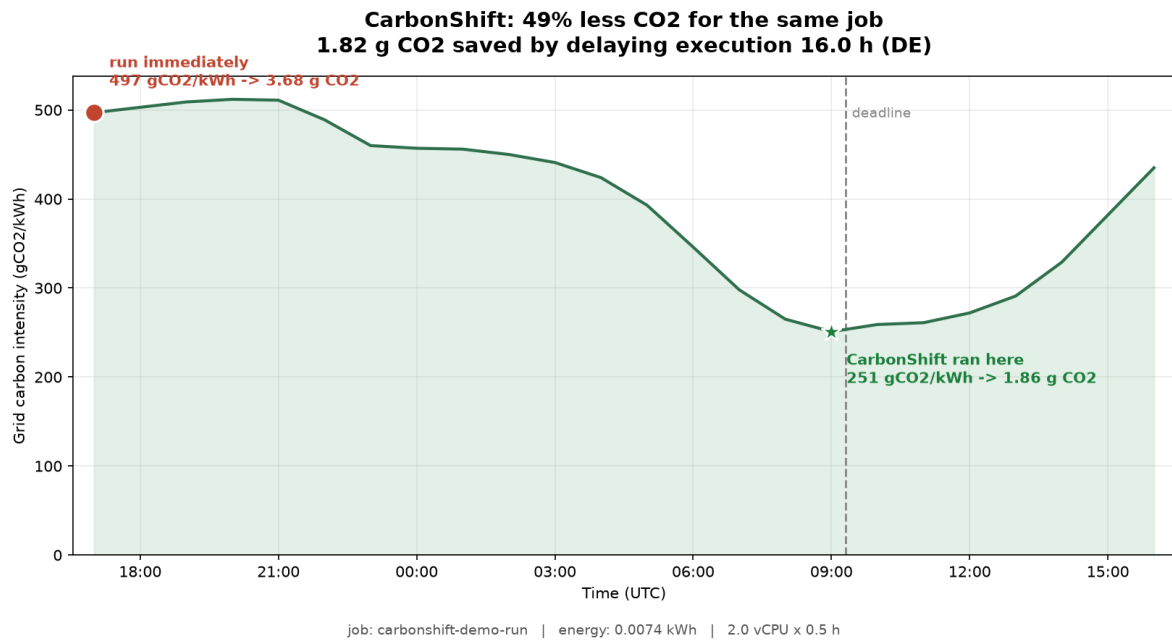


CarbonShift

A Carbon-Aware Job Scheduler for the Cloud

Complete Technical Report



Ramya Samudrala
B.Sc. Computer Science and Cloud Computing
International Conference Project Exhibition

Repository: github.com/Ramyasamudralaa/carbonshift | MIT License
Report generated 17 September 2026

1. Executive summary

CarbonShift delays a flexible cloud job until the cleanest hour the electricity grid will reach before that job's deadline, then runs it unchanged at that moment. The application code is never modified. Only the timing changes.

The system was built, deployed to AWS, and **verified end to end**: a real job was submitted, scheduled by AWS EventBridge Scheduler, and executed by ECS Fargate at the chosen instant, with its completion log captured in CloudWatch.

Measure	Result
Specification checks passed	8 of 8
Automated tests	177 passing , across 11 files
Python source	~7,290 lines across 11 components
Git commits	28, incremental
Real bugs found by live deployment	7, all fixed
Verified execution latency	21 seconds after the scheduled instant
Grid carbon range observed (DE)	251 to 512 gCO ₂ /kWh within 24 hours
Best single result	2.01 g CO ₂ , 67.9% , for a 17-hour delay
Cumulative across 10 scheduled runs	10.33 g CO ₂ , 34.2%

1.1 What was actually proven

- A carbon-intensity forecast can be fetched and parsed from a live commercial API, with every failure mode producing a clear, catchable error rather than a crash or bad data.
- The lowest-carbon hour inside a hard deadline is selected correctly, including the case where the globally cleanest hour falls outside the permitted window.
- AWS EventBridge Scheduler can be driven programmatically to fire a one-time ECS RunTask at a precise instant, and does so within seconds of the target.
- The container AWS ran produced a checksum byte-identical to the local run, proving the same deterministic workload executed in both places.
- The saving is real but modest in absolute terms and depends entirely on how variable the target grid is. This report states that plainly rather than obscuring it.

How to read this report

Sections 2 to 4 explain the idea and the shape of the system. Section 5 walks the code file by file. Sections 6 and 7 cover AWS infrastructure and how the pieces connect. Sections 8 to 10 are build history, bugs and evidence. Sections 11 to 13 are results, honest limitations, and how another developer would adopt it.

2. The idea

2.1 Carbon intensity is not constant

Carbon intensity is the mass of CO₂ emitted per unit of electrical energy consumed, in grams of CO₂ per kilowatt-hour (gCO₂/kWh). It is a property of the grid at a moment in time, not of the device drawing the power.

It varies because the generation mix varies. When wind and solar are abundant, more demand is met by zero-carbon sources and intensity falls. When they are not, the shortfall is met by gas and coal and intensity rises. On the German grid this produced a measured 24-hour range of 251 to 512 gCO₂/kWh during this project - a factor of 2.6 between the cleanest and dirtiest hour of a single day.

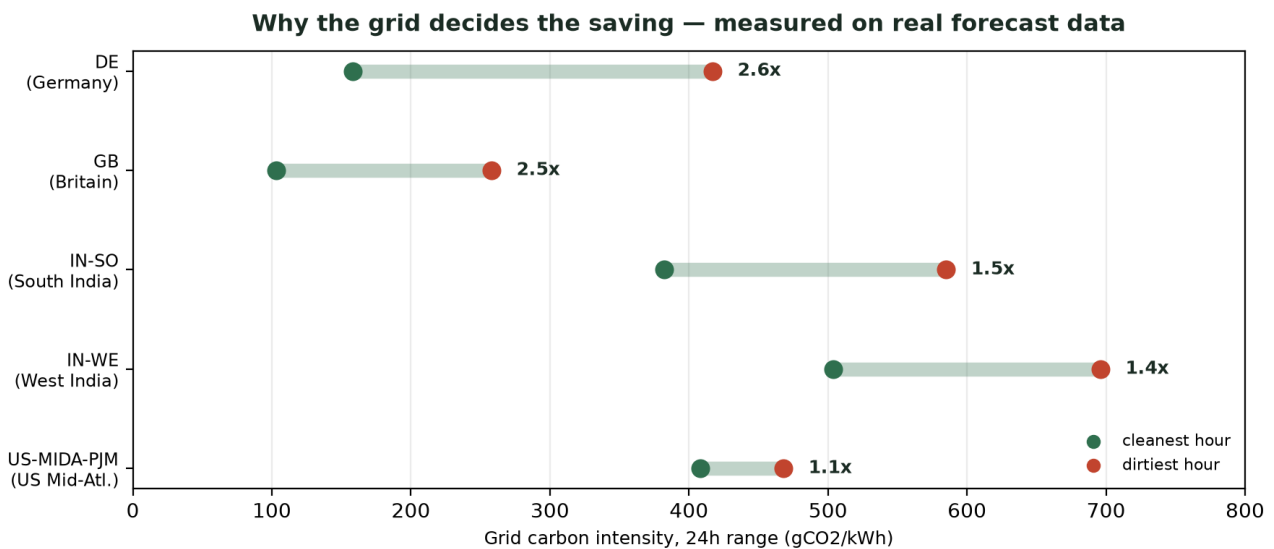


Figure 2.1 - 24-hour carbon-intensity range across five grid zones, measured on live Electricity Maps forecast data.

2.2 The waste

Cloud workloads are triggered with no awareness of this variation. A backup scheduled for 20:00 runs at 20:00, on whatever the grid happens to be burning. For genuinely time-flexible work - nightly ETL, report generation, batch processing, model training - the hour is arbitrary, and an arbitrary choice is on average worse than an informed one.

2.3 The carbon arithmetic

Emissions are the product of energy drawn and the carbon intensity of the grid supplying it:

```
emissions (gCO2) = energy (kWh) x carbon_intensity (gCO2/kWh)
energy (kWh)      = vCPU count x runtime (hours) x kWh per vCPU-hour
```

For the demo worker, at ~0.0074 kWh per vCPU-hour for typical cloud utilisation:

```
energy = 2 vCPU x 0.5 h x 0.0074 = 0.0074 kWh
```

```
at 497 gCO2/kWh -> 0.0074 x 497 = 3.68 g CO2 (run immediately)
```

```
at 251 gCO2/kWh -> 0.0074 x 251 = 1.86 g CO2 (run at the solar peak)
```

```
saving = 1.82 g CO2 = 49.5%
```

The energy term is identical in both cases. The same instructions execute for the same duration on the same hardware. Only the second term changes. That is the whole mechanism: **you cannot make the job cheaper, so you make the electricity cleaner.**

2.3.1 What is measured, and what is approximated

This distinction matters, and the first version of this report blurred it.

Quantity	Status
Grid carbon intensity (497, 251 gCO2/kWh)	Measured. Supplied by Electricity Maps for the grid zone concerned.
The hour the job ran	Measured. Recorded by AWS in CloudWatch.
That the same workload ran	Measured. Identical checksums.
Electricity the container drew	Approximated. 0.0074 kWh per vCPU-hour, a published figure for cloud compute at typical utilisation.

The percentage saving does not depend on the approximation at all. The energy term appears on both sides of the ratio and cancels:

$$\text{saving \%} = \frac{E \times I_{\text{baseline}} - E \times I_{\text{chosen}}}{E \times I_{\text{baseline}}} = \frac{I_{\text{baseline}} - I_{\text{chosen}}}{I_{\text{baseline}}}$$

$$49.5\% = (497 - 251) / 497 \quad \leftarrow \text{only measured quantities remain}$$

So 49.5% is computed entirely from measured grid data. Only the absolute gram figures carry the coefficient, and they are labelled as estimates wherever they appear.

Why the coefficient cannot simply be measured

AWS does not expose per-task power draw. There is no meter on a Fargate container, and no cloud provider offers one. Every carbon accounting tool therefore uses published coefficients for compute energy. This is standard practice in the field rather than a shortcoming of this implementation, though it does mean the absolute figures should be read as well-founded estimates.

2.4 Why this is not already solved

No major cloud provider - AWS, Azure or GCP - offers carbon-aware job scheduling as a native first-party service. It exists only as an architectural pattern developers assemble themselves from a third-party data provider plus the cloud's own scheduling primitives. Adjacent work exists: the Green Software Foundation publishes a Carbon Aware SDK that answers *when is the greenest time*, and carbon-aware scheduling is being added to the Java JobRunr framework. What is missing is a small, readable, open-source implementation that goes all the way to provisioning infrastructure and executing the job. That last mile is what CarbonShift covers.

3. Scope decisions

Scope was fixed before implementation and held throughout. The single use case is **one job, one deadline, one decision, one measurable result**.

3.1 Deliberately excluded

Not built	Reasoning
Multi-cloud support	One cloud done completely and verified beats three done partially. The decision logic is cloud-agnostic, so a port is contained - see section 13.3.
Web dashboard as a product feature	A job queue UI with accounts and CRUD would consume most of the effort and prove nothing about the carbon mechanism. See 3.4 for what was built instead.
User accounts / multi-tenancy	CarbonShift runs inside the operator's own AWS account. There is no server, so there is nothing to authenticate against.
Multiple concurrent jobs	A queue is a different engineering problem. One job proves the idea; scheduling many is an extension, not a prerequisite.
Modifying the workload's code	The central claim is that no application change is needed. Touching the workload would invalidate it.

3.2 One deliberate addition

A single behaviour was added beyond the original specification after live data exposed a flaw: **CarbonShift refuses to shift a job into an hour dirtier than running it immediately**. Documented in full in section 9.4. This was an explicit decision, taken consciously, not scope drift.

3.3 A distinction worth making: demo/dashboard.py

A live dashboard does exist, and it is worth being precise about why that is not a breach of the exclusion above.

What the specification rules out is a *product feature*: a job queue interface with accounts, state and CRUD. What was built is *presentation material*, which section 6 of the specification explicitly asks for. It is read-only, generated from decisions already recorded on disk, holds no state of its own, and cannot change anything. It therefore lives in `demo/` rather than `src/`, which is the line between the two.

It exists because a one-time schedule deletes itself the moment it fires, leaving nothing visual to point at afterwards. It is a window onto the run records, not an application.

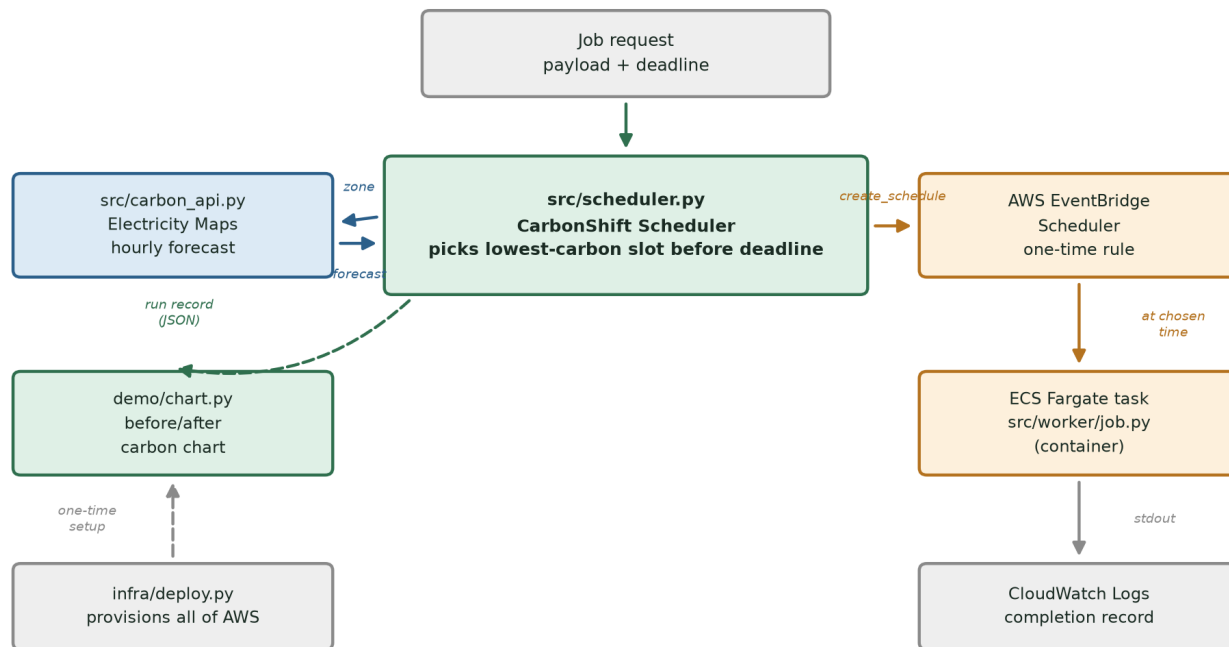
3.4 One structural addition

A `tests/` directory was added, which the specified repository layout does not list. The specification requires eight verification checks but does not say where tests should live. No other top-level directory was added and nothing was renamed.

4. Architecture

Five components. Each has exactly one responsibility, and no component reaches into another's.

CarbonShift architecture — five components, one responsibility each



Green = portable decision logic | Amber = AWS-specific execution

Figure 4.1 - Component architecture. Green components contain no cloud-provider code; amber components are AWS-specific.

Component	File	Responsibility
Carbon data client	src/carbon_api.py	Fetch and parse the hourly forecast for a grid zone. Contains no AWS code.
Scheduling logic	src/scheduler.py	Select the lowest-carbon hour before the deadline; register the one-time EventBridge rule.
Worker job	src/worker/	The containerised demo workload. Deterministic, so a scheduled run is verifiable.
Infrastructure	infra/deploy.py	One-time boto3 provisioning of every AWS resource the scheduler depends on.
Results chart	demo/chart.py	Render the before/after carbon comparison as a single static image.

Six further tools were built on that core once it worked. They add no new cloud integration; each reads what the five components already produce.

Tool	File	What it does
Menu	carbonshift.py	Everything as numbered options, no flags to remember.
Setup wizard	setup.py	Checks prerequisites, tests the API token live, provisions AWS and writes every setting itself.
Job scanner	src/scanner.py	Reads a crontab or your AWS schedules and classifies which jobs are shiftable.
Run history	src/history.py	Reads back every decision, cross-checked against CloudWatch.
Carbon report	src/report.py	A filable document with methodology, limitations and per-job verification.

Tool	File	What it does
Live dashboard	demo/dashboard.py	One self-contained page: curve, countdown, running total. Works offline.
Doctor	src/doctor.py	Diagnoses a broken setup in one pass, each failure paired with its fix.

4.2 The separation that matters

Decision logic and execution are deliberately not entangled. Counting cloud-provider references per file:

File	AWS references	Portable?
demo/chart.py	0	Yes
src/carbon_api.py	1 (a docstring)	Yes
src/worker/job.py	1 (sample data)	Yes
src/scheduler.py	47	Decision functions yes, scheduling no
infra/deploy.py	73	No - entirely AWS

The four functions that make the actual decision - `pick_best_slot`, `baseline_slot`, `estimate_energy_kwh` and `emissions_grams` - contain zero references to `boto3`, `ECS` or `EventBridge`. This was verified by AST inspection of the function bodies, not by eye.

4.3 Sequence of a single job

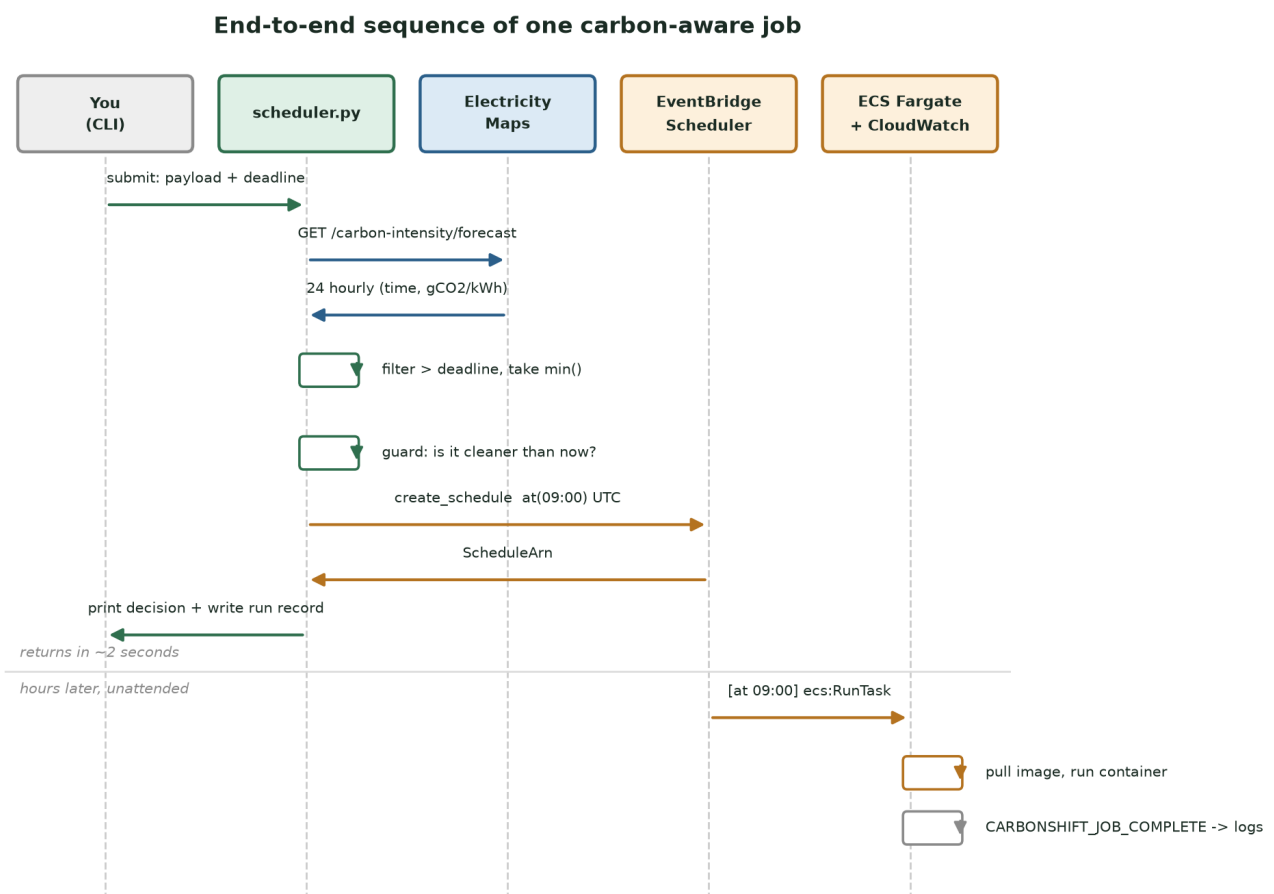


Figure 4.2 - The full lifecycle. Everything above the divider completes in about two seconds; everything below happens hours later, unattended.

The critical property is that the two halves are decoupled. Once the schedule is registered, the submitting machine can be switched off. Execution is AWS's responsibility, not the client's - which is precisely why EventBridge Scheduler was chosen over a local cron job or a long-running process.

5. Code walkthrough

This section covers every source file: what it does, its key functions, and why it is written the way it is.

5.1 src/carbon_api.py - the forecast client

Responsibility: given a grid zone, return an hourly carbon-intensity forecast. It knows nothing about AWS, scheduling, or the job being run.

The data model

```
@dataclass(frozen=True)
class ForecastEntry:
    timestamp: datetime          # timezone-aware, always normalised to UTC
    carbon_intensity: float      # gCO2 per kWh
```

frozen=True makes entries immutable - a forecast point that silently changed after the decision was made would be a difficult bug. Timestamps are normalised to UTC on parse so no comparison anywhere else has to worry about offsets.

Errors are a taxonomy, not an afterthought

```
CarbonApiError          # base - one except clause catches everything
CarbonApiConfigError    # missing API key, blank zone, bad hours
CarbonApiTimeoutError   # the API did not respond in time
CarbonApiConnectionError # the API could not be reached at all
CarbonApiAuthError      # 401/403 - wrong key, or zone not on your plan
CarbonApiResponseError   # HTTP error, non-JSON body, unusable payload
```

Six exception types rather than one because the caller's response differs in each case, and because the message can then be specific. The auth error, for example, names the single most likely cause:

```
if response.status_code in (401, 403):
    raise CarbonApiAuthError(
        f"Carbon API rejected the request for zone '{zone}' "
        f"(HTTP {response.status_code}). Either {API_KEY_ENV_VAR} is wrong, "
        "or your plan does not include this zone -- the Electricity Maps "
        "free tier is normally limited to a single home zone."
    )
```

Two deliberate parsing decisions

Null intensities are skipped, not zeroed. The provider emits null for hours it has no model output for. Treating null as 0 would make it look like the cleanest hour available and the scheduler would pick it every time - a silent, serious error.

An all-null forecast raises rather than returning an empty list. An empty list could be misread by a caller as *no clean hours available*, when the truth is *we have no data*. Those demand different responses.

```
if intensity is None:
    continue          # skip, never substitute a value
...
if not entries:
    raise CarbonApiResponseError(
        "Carbon API returned a forecast with no usable carbon-intensity "
        "values. Check that your plan covers this zone."
    )
```

The API key never reaches the code

The key is read from the environment at call time, and checked *before* any network call is made, so a missing key fails fast and locally rather than as a confusing 401. It is sent as a header, never as a query parameter, because

URLs end up in logs and browser history.

5.2 src/scheduler.py - the decision engine

Responsibility: pick the lowest-carbon hour before the deadline and register the AWS schedule. This is the heart of the project.

Slot selection - the core algorithm

```
def pick_best_slot(forecast, deadline, earliest=None):
    eligible = [e for e in forecast if e.timestamp <= deadline]
    if earliest is not None:
        eligible = [e for e in eligible if e.timestamp >= earliest]

    if not eligible:
        raise NoSlotBeforeDeadlineError(...)

    # min() keeps the first of any ties, and the list is chronological,
    # so a tie resolves to the earliest clean hour.
    return min(eligible, key=lambda entry: entry.carbon_intensity)
```

The algorithm itself is simple, and the report should not pretend otherwise - it is a filter followed by a minimum. The engineering is in the three properties around it:

- **The deadline is an exclusion applied before the minimum is taken**, not a tie-breaker afterwards. If the globally cleanest hour is outside the window it is discarded entirely, never chosen and then corrected.
- **The earliest bound keeps the scheduler out of the past**. The scheduling path passes now + 120 s, because EventBridge rejects times in the past and a schedule created for an instant that has already elapsed would silently never fire.
- **Ties resolve to the earliest clean hour**. If two hours are equally clean there is no carbon reason to prefer the later one, and finishing sooner is strictly better.

The baseline, and the guard it necessitated

The baseline is the earliest forecast hour - the answer to *what would have happened with no CarbonShift*. But that hour is excluded from eligibility as soon as it is in the past, so the minimum of what remains can be worse than the baseline:

```
if require_improvement and chosen.carbon_intensity >= baseline.carbon_intensity:
    raise ShiftNotWorthwhileError(
        "Running immediately is already the cleanest option. ..."
    )
```

Ties are refused as well: a delay that saves nothing is still a delay. The CLI treats this as exit code 0, because *the cleanest moment is now* is a correct answer, not a malfunction. Full history in section 9.4.

Building the EventBridge request

```

target_input = {
    "TaskDefinition": config["task_definition_arn"],
    "Cluster": config["cluster_arn"],
    "LaunchType": "FARGATE",
    "Count": 1,
    "NetworkConfiguration": {
        "AwsVpcConfiguration": {
            # PascalCase - see section 9.3
            "Subnets": config["subnets"],
            "AssignPublicIp": config["assign_public_ip"],
            "SecurityGroups": config["security_groups"],
        }
    },
    "Overrides": {"ContainerOverrides": [{
        "Name": "carbonshift-worker",
        "Environment": [
            {"Name": "CARBONSHIFT_JOB_PAYLOAD", "Value": job_payload},
            {"Name": "CARBONSHIFT_SCHEDULED_FOR", "Value": run_at.isoformat()},
        ]
    }]}
}

request = {
    "Name": schedule_name,
    "ScheduleExpression": f"at({utc_no_offset})", # at(2026-09-11T09:00:00)
    "ScheduleExpressionTimezone": "UTC",
    "FlexibleTimeWindow": {"Mode": "OFF"}, # fire exactly, not "within 15 min"
    "ActionAfterCompletion": "DELETE", # self-clean one-time schedules
    "Target": {
        "Arn": "arn:aws:scheduler::aws-sdk:ecs:runTask",
        "RoleArn": config["role_arn"],
        "Input": json.dumps(target_input),
    },
}

```

Field	Why this value
at(...) with no offset	EventBridge one-time expressions reject an offset suffix. The timestamp is converted to UTC and the tzinfo stripped, with the zone declared separately.
ScheduleExpressionTimezone: UTC	Removes any ambiguity around daylight saving. Everything internal is UTC end to end.
FlexibleTimeWindow: OFF	The whole point is to run in a specific hour. A flexible window would let AWS drift the execution into a dirtier one.
ActionAfterCompletion: DELETE	One-time schedules delete themselves after firing, so repeated tests do not accumulate and the account stays clean.
aws-sdk:ecs:runTask	The EventBridge <i>universal target</i> , which can call any AWS SDK action directly - no Lambda glue function needed between the schedule and ECS.

Lazy boto3 import

boto3 is imported *inside* the scheduling function, not at module top. This means the entire decision path - and its tests - runs on a machine with no AWS SDK installed, which keeps the unit tests fast and genuinely offline.

5.3 src/worker/ - the workload

Responsibility: be the job that gets scheduled. Its purpose is to prove the mechanism fired, not to be a realistic production workload.

It is deliberately **deterministic**: it aggregates a fixed dataset and checksums the result, so the same run always produces the same output. That is what made it possible to prove the container AWS ran was the same one that ran locally - the checksums match exactly.

```
def process(dataset):
    total_kwh = round(sum(row["kwh"] for row in dataset), 6)
    by_region = {row["region"]: row["kwh"] for row in dataset}
    canonical = json.dumps({"total_kwh": total_kwh, "by_region": by_region},
                           sort_keys=True)
    checksum = hashlib.sha256(canonical.encode("utf-8")).hexdigest()[:16]
    return {"rows_processed": len(dataset), "total_kwh": total_kwh,
           "checksum": checksum}
```

sort_keys=True matters: without it, dictionary ordering could vary between Python builds and the checksum would stop being a reliable identity check.

Completion is announced on a single greppable line, so verification after the fact is one command:

```
log("CARBONSHIFT_JOB_COMPLETE " + json.dumps({...}, sort_keys=True))
```

The Dockerfile

```
FROM python:3.12-slim

ENV PYTHONUNBUFFERED=1 \
    PYTHONDONTWRITEBYTECODE=1 \
    WORKER_RUN_SECONDS=5

WORKDIR /app
COPY job.py /app/job.py

RUN useradd --create-home --uid 10001 carbonshift
USER carbonshift

ENTRYPOINT ["python", "/app/job.py"]
```

Choice	Why
python:3.12-slim	Smallest official image that still has a full CPython. Faster pulls mean less time between the scheduled instant and actual execution.
PYTHONUNBUFFERED=1	Without it Python buffers stdout when not attached to a terminal, so logs would only reach CloudWatch at exit - or not at all if the task were killed. This is the single most important line for observability.
Standard library only	No pip install layer, so the image builds in seconds and has a minimal dependency surface.
Non-root user	Fargate does not require root for this workload, so it does not get it.

5.4 infra/deploy.py - provisioning

Responsibility: create every AWS resource the scheduler depends on, then print the exact configuration block to paste into `.env`.

Every step is **idempotent** - it tolerates the resource already existing and reuses it. The pattern throughout is to attempt creation and treat the already-exists error as success:

```
try:
    response = ecr.create_repository(repositoryName=ECR_REPOSITORY_NAME)
    detail("created")
except ClientError as exc:
    if _error_code(exc) != "RepositoryAlreadyExistsException":
        raise _explain(exc, "Creating the ECR repository")
    response = ecr.describe_repositories(repositoryNames=[ECR_REPOSITORY_NAME])
    detail("already exists, reusing")
```

There is one honest exception, documented in the module docstring: ECS has no update-in-place for task definitions, so re-running registers a new revision. The script says so and prints the new ARN rather than pretending otherwise.

Errors name the missing permission

```
def _explain(exc, what):
    code = _error_code(exc)
    hints = {
        "AccessDenied": "Your AWS identity lacks the IAM permission for this call.",
        "InvalidClientTokenId": "Your AWS credentials are invalid or expired.",
        "OptInRequired": f"Region {AWS_REGION} is not enabled for this account.",
    }
    return DeployError(f"{what} failed in {AWS_REGION}. AWS said [{code}]: {message}"
                      + (f" -- {hints.get(code, '')}" if code in hints else ""))
```

The specification requires that provisioning failures identify which permission or resource is missing. A raw `botocore` traceback does not do that; this does.

5.5 The tools built on top

Each of these reads what the core already produces. None adds cloud integration.

src/doctor.py - catching silent failures

Scheduling is asynchronous: the job fires hours later on a machine nobody is watching. So the failures that matter are the quiet ones. If the worker image is missing from ECR, `scheduler.py` still books the job perfectly, because booking has no reason to check. ECS then fails to pull it at 03:00, the container never starts so there is no log, and the schedule has already deleted itself. **Nothing anywhere reports a problem.**

The doctor checks the machine, the API key against the live service, every AWS resource the configuration points at, whether the image is really in ECR, Docker, and whether past scheduled runs produced logs. Each failure prints the command that fixes it.

src/scanner.py - which jobs are worth shifting

CarbonShift can move a job you already know is flexible. Deciding that for forty `crontab` entries means reading each one and judging whether anything is waiting on its output.

Two classifiers. The default uses rules over the command text and the run frequency, needs no key and costs no energy; frequency is the strongest single signal, since anything running more often than hourly is keeping something alive rather than batching. The other has an LLM read each job, and falls back to the rules if the key is absent or the reply unusable.

Why the LLM does not choose the hour

Choosing the cleanest hour is `min()` over 24 numbers. An LLM there would be slower, cost money, and give different answers on different runs. Judging whether a job can wait six hours is the part worth an LLM. Classification also runs once per job definition rather than once per execution, so its energy cost amortises across every future run - an LLM call that cost more carbon than the shift saved would defeat the purpose.

src/report.py - evidence rather than a number

A running total on a terminal is not evidence. An organisation reporting emissions needs a document: the period, the jobs, the figures, the method, and an honest account of what the figures are not.

Three rules it will not break. Previews never count, because they were never scheduled. With verification enabled, jobs with no CloudWatch log are excluded from every figure and the report declares how much was therefore not claimed. Without verification, the report says so on its face.

src/history.py and demo/dashboard.py

A one-time schedule deletes itself when it fires, so afterwards there is nothing in the console to look at. Both read the decisions recorded at the time. The dashboard builds a single self-contained HTML file with the carbon curve, a live countdown and the running total, with no server and no network once open.

5.6 demo/chart.py - the visual

Responsibility: render one static image showing the carbon curve, the naive run-immediately point, the point CarbonShift chose, the deadline, and the saving.

It reads the run record JSON that the scheduler wrote, so the chart is generated from the actual recorded decision rather than recomputed - the picture cannot drift from what the system really did. `matplotlib.use("Agg")` is set before pyplot is imported so it renders headlessly with no display attached.

The `--actual-run` flag takes the verified execution timestamp from CloudWatch and draws it as a separate dotted line, so the chart distinguishes *when we intended to run* from *when it actually ran*.

6. AWS infrastructure

Eight resources, all created by `infra/deploy.py` in a single run against a fresh AWS account.

Resource	Name	Purpose
ECR repository	carbonshift-worker	Stores the worker image where Fargate can pull it. The image cannot live on the laptop - Fargate runs in a Frankfurt datacentre and cannot reach it.
ECS cluster	carbonshift-cluster	Logical grouping the Fargate task runs in. With Fargate there are no servers to manage; the cluster is essentially a namespace.
CloudWatch log group	/ecs/carbonshift-worker	Destination for container stdout. 14-day retention so test runs expire on their own.
ECS service-linked role	AWSServiceRoleForECS	Lets the ECS service act on the account's behalf. Absent on fresh accounts - see 9.1.
Task execution role	carbonshift-ecs-execution-role	Assumed by ECS to pull the image from ECR and write logs. Carries the AWS-managed AmazonECSTaskExecutionRolePolicy.
Scheduler role	carbonshift-scheduler-role	Assumed by EventBridge Scheduler to call <code>ecs:RunTask</code> . Scoped to this task family in this cluster only.
Task definition	carbonshift-worker:1	The blueprint: 2 vCPU, 4 GB, awsvpc networking, image URI, log configuration.
Networking	default VPC, 3 subnets	Reuses the account default VPC rather than creating one - cheaper, simpler, and nothing here needs network isolation.

6.1 The IAM trust model

Two roles, not one, and the separation is deliberate.

IAM trust model — who is allowed to do what

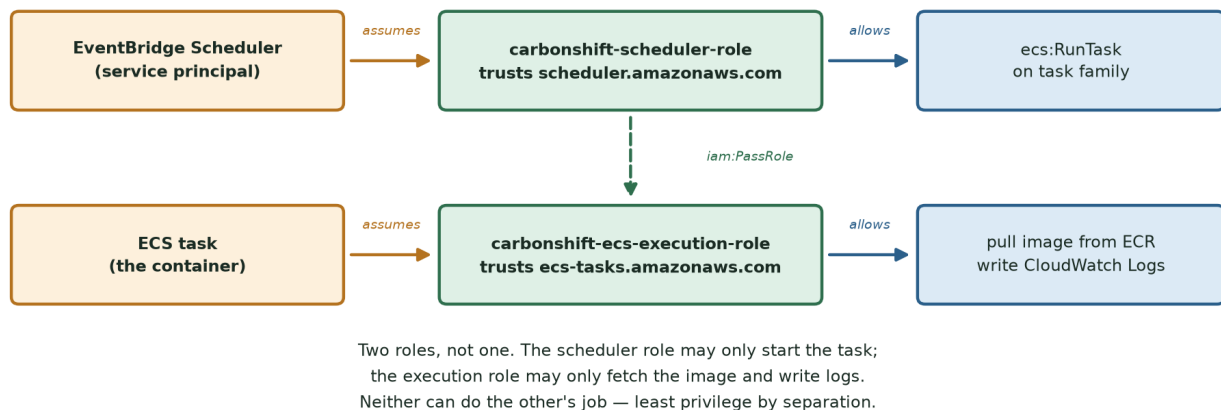


Figure 6.1 - Two roles with distinct trust relationships. Neither can perform the other's function.

A **trust policy** answers *who may assume this role*. A **permission policy** answers *what may they then do*. Both are needed; either alone is useless.

```
# Trust policy - who may become this role
{"Effect": "Allow",
 "Principal": {"Service": "scheduler.amazonaws.com"},
 "Action": "sts:AssumeRole",
 "Condition": {"StringEquals": {"aws:SourceAccount": account_id}}}

# Permission policy - what it may then do
{"Effect": "Allow",
 "Action": "ecs:RunTask",
 "Resource": ["arn:aws:ecs:REGION:ACCT:task-definition/carbonshift-worker:*"],
 "Condition": {"ArnLike": {"ecs:cluster": ".../cluster/carbonshift-cluster"}}}
```

- The `aws:SourceAccount` condition prevents the confused-deputy problem: another AWS account cannot induce EventBridge to assume this role on their behalf.
- `ecs:RunTask` is scoped to this task family *and* this cluster. The role cannot start arbitrary containers in the account.
- `iam:PassRole` is restricted to the execution role and further conditioned on the target service being `ecs-tasks` - without this condition it would be a privilege-escalation path.

6.2 Why EventBridge Scheduler

Alternative	Why it was rejected
Local cron	Requires the submitting machine to stay on for up to 24 hours. Defeats the purpose of a cloud scheduler.
EventBridge Rules (classic)	Designed for recurring events; one-time scheduling is awkward and rules are not self-deleting.
Lambda + DynamoDB + polling	Far more moving parts, and a polling loop that itself consumes energy to save energy.
Step Functions wait state	Workable, but a whole state machine to express one delay is heavier than the problem warrants.
EventBridge Scheduler	Purpose-built for one-time schedules, calls the ECS API directly via universal targets, self-deletes after firing, and costs effectively nothing.

7. How it connects, end to end

The complete path of one job, with nothing skipped.

#	What happens	Where
1	You run the CLI with a payload and a deadline.	Your laptop
2	<code>load_dotenv()</code> runs at import, populating the AWS configuration constants before they are evaluated.	<code>scheduler.py</code>
3	The AWS region is mapped to an Electricity Maps zone (eu-central-1 to DE), or <code>CARBON_ZONE</code> overrides it.	<code>scheduler.py</code>
4	HTTPS GET to the forecast endpoint with the API key as a header. 10-second timeout.	<code>carbon_api.py</code>
5	24 hourly points parsed, nulls skipped, timestamps normalised to UTC, sorted chronologically.	<code>carbon_api.py</code>
6	Entries after the deadline are discarded. Entries before now+120s are discarded.	<code>pick_best_slot</code>
7	<code>min()</code> over what remains, by carbon intensity.	<code>pick_best_slot</code>
8	Guard: if the winner is not cleaner than the baseline, refuse and stop.	<code>schedule_job</code>
9	The decision is serialised to <code>runs/<job-id>.json</code> for the chart to read later.	<code>write_run_record</code>
10	<code>scheduler.create_schedule()</code> registers a one-time rule with the ECS RunTask payload embedded as JSON.	AWS API call
11	The decision and the ScheduleArn are printed. The CLI exits. Total time ~2 seconds.	Your laptop
12	<i>Hours pass. The laptop can be switched off.</i>	-
13	At the scheduled instant EventBridge assumes the scheduler role and calls <code>ecs:RunTask</code> .	AWS
14	ECS assumes the execution role, pulls the image from ECR, attaches an ENI in a default-VPC subnet.	AWS
15	The container starts. Environment overrides carry the payload and the intended time.	Fargate
16	The worker processes its dataset and prints <code>CARBONSHIFT_JOB_COMPLETE</code> . Unbuffered, so it streams live.	Fargate
17	The awslogs driver ships stdout to the CloudWatch log group.	AWS
18	The container exits 0. Fargate tears down. The schedule deletes itself.	AWS
19	Later, you run <code>demo/chart.py</code> with the verified timestamp to render the result.	Your laptop

The measured numbers for step 13 to 16

Schedule fired at 17:00:00 UTC. Container start logged at 17:00:21 UTC. That 21-second gap is Fargate cold start - provisioning the ENI and pulling the image. For a scheduler whose decisions are hour-granular, 21 seconds is irrelevant, but it is worth knowing that sub-minute precision is not on offer.

8. Build chronology

What was done, in order, including what did not work the first time. The git history mirrors this: 28 commits, each a single coherent change.

Phase	What happened	Outcome
Scaffold	Repository structure per specification, MIT licence, requirements, .env.example, .gitignore excluding .env from the first commit.	Clean
Forecast client	carbon_api.py with a six-way error taxonomy and strict payload validation.	Clean
Decision engine	scheduler.py: slot selection, carbon maths, EventBridge request construction, run records.	Clean
Worker	job.py plus Dockerfile, standard library only.	Clean
Provisioning	deploy.py: ECR, cluster, log group, two IAM roles, task definition, network discovery.	Clean
Chart	chart.py reading the run record and rendering the comparison.	Clean
Test suite	58 tests covering everything verifiable without AWS.	All passing
Documentation	README with problem, solution, architecture, setup, verification.	Clean
Live API check	First call against real Electricity Maps. The token worked for all five zones tested.	Check 1 passed
Docker check	Image built and ran standalone, exit 0, checksum 8cca50fc7eb250ec.	Check 5 passed
First provisioning	Failed: <i>Unable to assume the service linked role.</i>	Bug 1
Second provisioning	After the fix, all eight resources created.	Clean
Image push	Tagged and pushed to ECR.	Clean
First schedule attempt	Failed: <i>Missing AWS configuration</i> despite a fully populated .env.	Bug 2
Second attempt	Failed: <i>field aws_vpcConfiguration is not supported.</i>	Bug 3
Third attempt	Schedule created, ENABLED, self-deleting.	Clean
Waiting	A watcher polled CloudWatch for 40 minutes and reported nothing. The diagnosis was wrong.	Bug 4
Verification	The task had run perfectly at 17:00:21, exit 0, log present.	Checks 6 and 7
Guard added	Live data showed a shift could increase emissions.	72 tests
Publication	Secret scan of full history, account id redacted, pushed public.	Clean
Usability round	setup wizard, run history, doctor, interactive menu, live dashboard.	125 tests
Compliance reports	Filable document with methodology and verification.	Bug 5 found in its own methodology
Job scanner	Heuristic and LLM classifiers over crontabs and AWS schedules.	Bugs 6 and 7
Setup hardening	Prerequisite checks for Docker and the AWS CLI.	Bug 8
not-before bound	Windows can now be bounded at both ends.	177 tests

9. Bugs found and fixed

Eight defects. **None were findable by unit testing alone** - each required contact with live infrastructure, a real machine, or real forecast data. This is the most instructive section of the report, because it is the part no amount of planning would have predicted.

9.1 The ECS service-linked role does not exist on fresh accounts

Symptom	<code>InvalidParameterException: Unable to assume the service linked role on the first create_cluster call.</code>
Root cause	AWS creates <code>AWSServiceRoleForECS</code> implicitly the first time you make a cluster <i>in the console</i> . An account driven purely through the API never triggers that, so the role is simply absent.
Why tests missed it	It is a property of the AWS account, not the code. Only a genuinely fresh account reveals it.
Fix	An explicit step before cluster creation, treating IAM's <code>InvalidInput</code> 'has been taken' as success so it stays idempotent.
Impact	Would have broken for every open-source user with a new AWS account, at the very first step.

9.2 .env loaded after the configuration constants

Symptom	Missing <code>AWS</code> configuration despite <code>.env</code> containing every one of those values, correctly spelled.
Root cause	A module-initialisation ordering error. The constants are module-level and evaluated at import, but <code>load_dotenv()</code> ran inside <code>main()</code> , later. The constants captured empty strings.
Why tests missed it	Every test injected configuration directly, so none exercised the real import-then-read path.
Fix	Move the load above the constants, and pin the ordering with a test that reads the source and asserts it.
Impact	Would have broken for every user, with an error that actively misled.

9.3 The universal target requires PascalCase

Symptom	The field <code>'awsvpcConfiguration'</code> is not supported by api <code>'runTask'</code> .
Root cause	The ECS API uses camelCase and every boto3 example follows it. But the EventBridge universal target constructs a raw AWS SDK request, which is PascalCase throughout.
Why tests missed it	The test asserted the request was well-formed by our own definition. Only AWS knows AWS's schema.
Fix	One capital letter. The test now asserts the lowercase spelling is absent.
Impact	Complete failure to schedule anything.

9.4 CarbonShift could increase emissions

The most interesting of the eight, because the code was doing exactly what the specification said and the specification was incomplete.

```
run immediately   : 2026-09-10T16:00:00+00:00 at 462 gCO2/kWh -> 3.42 g CO2
CarbonShift picks : 2026-09-10T17:00:00+00:00 at 497 gCO2/kWh -> 3.68 g CO2
CO2 saved         : -0.26 g (-7.6%)
```

Root cause	The baseline is the earliest forecast hour, but that hour leaves the eligible set the moment it is in the past. So the minimum of what remains can be dirtier than the baseline.
Why tests missed it	Every fixture had a clean hour inside the window. The scenario needed a monotonically rising evening curve, which live data supplied and imagination had not.
Fix	Refuse to shift when the best eligible hour is no cleaner than the baseline. Ties refused too.
Impact	A carbon-aware scheduler that increases emissions is worse than none. A correctness bug against the project's purpose, not its written spec.

Why this counts as engineering rather than a patch

An implementation that satisfies the letter of a specification while defeating its purpose is still wrong. The fix was made as an explicit, recorded decision, with an opt-out for anyone who must delay regardless of carbon, rather than silently changing behaviour.

9.5 The compliance report contradicted its own method

Symptom	The methodology section stated that unverified runs were excluded from the totals. The arithmetic included them.
Root cause	The text was written describing intended behaviour; the code summed everything.
Why it matters	A compliance document whose stated method disagrees with its own figures is the first thing an auditor finds.
Fix	Behaviour changed to match the statement. With --verify, unprovable runs are excluded from every figure and the report declares how much was not claimed.

9.6 and 9.7 Two parser faults in the job scanner

Prose parsed as a job	A line of English in a crontab has six or more tokens, so it parsed as a schedule plus a command and produced a job with a nonsense frequency. Fixed by validating that the first five fields are actually cron fields.
AWS wildcards misread	AWS cron expressions write '?' where standard cron writes '*'. Treating '?' as a literal value made daily jobs look weekly, understating their annual saving by a factor of seven.

Both were caught by tests written at the same time as the feature, which is the distinction between these two and the earlier ones.

9.8 Docker: 'not installed' reported as 'not running'

Symptom	A user without Docker Desktop was told ' <i>Docker is not running. Start Docker Desktop.</i> ' with no download link anywhere.
Root cause	One check returned a boolean. Missing and stopped collapsed into the same answer, and the message assumed the second.
Fix	A three-state check: missing, stopped, running. Missing gets a download link, stopped gets an instruction to start it. Setup also now reports Docker and the AWS CLI up front rather than discovering them mid-provisioning.
Impact	Sent someone looking for an application they had never installed.

9.9 A wrong diagnosis, recorded deliberately

After the first schedule fired, a watcher polled CloudWatch for 40 minutes and found nothing. The conclusion reported at the time was that the job had not run.

That conclusion was wrong. The job had run perfectly at 17:00:21. The watcher was broken: Git Bash on Windows rewrites a leading slash into a Windows path, so the log group name never reached the AWS CLI. Every query silently returned nothing.

It is recorded because the lesson generalises: **an absence of evidence was treated as evidence of absence.** Checking whether the schedule had self-deleted, and whether an ECS task existed, would have found the truth in thirty seconds.

10. Testing and verification

10.1 The eight specification checks

#	Check	How verified	Status
1	Fetches and parses a real live forecast	Live call to Electricity Maps, 24 points for DE	PASS
2	Handles API failure and timeout without crashing	Automated: timeouts, connection errors, 401/403, 500, non-JSON	PASS
3	Excludes forecast timestamps after the deadline	Automated, including when the global minimum is outside	PASS
4	Identifies the true minimum-carbon slot	Automated, including shuffled input and tie-breaking	PASS
5	Worker image builds and runs standalone	docker run on Docker 29.7.2, exit 0	PASS
6	EventBridge rule fires the Fargate task	Live: fired 17:00:00, task started 17:00:21	PASS
7	Completion log visible in CloudWatch	Live: retrieved from the log stream	PASS
8	Chart renders correctly from a real run	Automated render plus human inspection	PASS

10.2 The automated suite

File	Tests	Focus
test_scheduler.py	42	Deadlines, minimum selection, tie-breaking, carbon maths, EventBridge request shape, the refusal guard, the not-before bound, .env ordering
test_scanner.py	37	Job classification, weighted towards never calling a latency-sensitive job shiftable
test_carbon_api.py	19	Parsing, six error types, null handling, header not query parameter
test_report.py	17	Period selection, verification splitting, arithmetic, CSV, markup escaping
test_doctor.py	16	Problem detection, fix messages, exit codes
test_dashboard.py	10	Countdown versus done, totals, injection safety
test_menu.py	10	Readiness logic, status line, scheduling guard
test_setup.py	10	Docker missing versus stopped, download links
test_deploy.py	6	Service-linked role branches, error quality
test_chart.py	6	Rendering from a real record, malformed input
test_worker.py	4	Determinism, greppable completion line

177 tests, all passing, in under two seconds. They require no AWS credentials and no network, because boto3 is imported lazily and every external call is injected.

10.3 What the tests deliberately do not cover

Being honest about coverage is more useful than claiming completeness:

- **Real AWS API contracts.** Tests assert our request is well-formed by our own definition. Only AWS knows AWS's schema, which is how bug 9.3 survived.

- **Account-level state.** Service-linked roles, region opt-in, quotas. Bug 9.1 lived here.
- **Timing behaviour.** Whether EventBridge fires on time is unmockable; it was verified by observation.
- **Forecast accuracy.** CarbonShift trusts Electricity Maps. A wrong forecast produces a wrong decision and nothing here detects it.
- **Long-running behaviour.** Nothing tests what happens over weeks of operation, because nothing has yet run for weeks.

11. Results

11.1 The mechanism, verified end to end

On 10 September a job was submitted, scheduled, and executed with no human involvement between submission and completion.

Submitted	16:33 UTC, one-hour deadline
Chosen slot	17:00:00 UTC
Task started	17:00:21 UTC , 21 seconds after target (Fargate cold start)
Exit code	0
Checksum	8cca50fc7eb250ec, identical to the local docker run
Schedule afterwards	Self-deleted, as configured

The single strongest piece of evidence

The checksum produced by the container AWS ran in Frankfurt is byte-identical to the one produced by docker run on the development laptop. The same deterministic workload demonstrably executed in both places. That is what distinguishes a working scheduler from a drawing of one.

11.2 The carbon results

Ten jobs have been scheduled and executed since. Every figure below comes from the recorded decision for that run.

Executed (UTC)	Job	Before	After	Saved	%
10 Sep 17:00	carbonshift-e2e-test	462	497	-0.26 g	-7.6
11 Sep 05:00	carbonshift-demo-run	497	393	+0.77 g	+20.9
11 Sep 09:00	carbonshift-demo-run	497	251	+1.82 g	+49.5
16 Sep 11:00	ramya-proof-test	252	236	+0.12 g	+6.3
17 Sep 12:00	nightly-backup	399	128	+2.01 g	+67.9
17 Sep 13:00	etl-warehouse-load	399	150	+1.84 g	+62.4
17 Sep 14:00	database-vacuum	393	192	+1.49 g	+51.1
17 Sep 15:00	monthly-billing-report	399	245	+1.14 g	+38.6
17 Sep 16:00	search-reindex	393	283	+0.81 g	+28.0
17 Sep 23:00	overnight-data-export	393	313	+0.59 g	+20.4

Cumulative: 10.33 g CO2 avoided, a 34.2% reduction across all ten runs.

What the spread actually shows

The percentages fall steadily down the table, and that is the most useful finding in this report. Each job was given a different window, and each found the cleanest hour it was *allowed* to reach. The backup could run in the afternoon solar peak and saved 67.9%. The overnight export could not start until after 21:00, when the German grid is dirty whatever you do, so it saved 20.4%.

The saving is a function of flexibility, not of the tool. CarbonShift cannot create clean hours. It can only reach the ones your deadline permits.

The first row is negative, and is kept deliberately. It was a mechanism test on a one-hour deadline during an evening ramp, before the refusal guard existed. It is the run that caused the guard to be written.

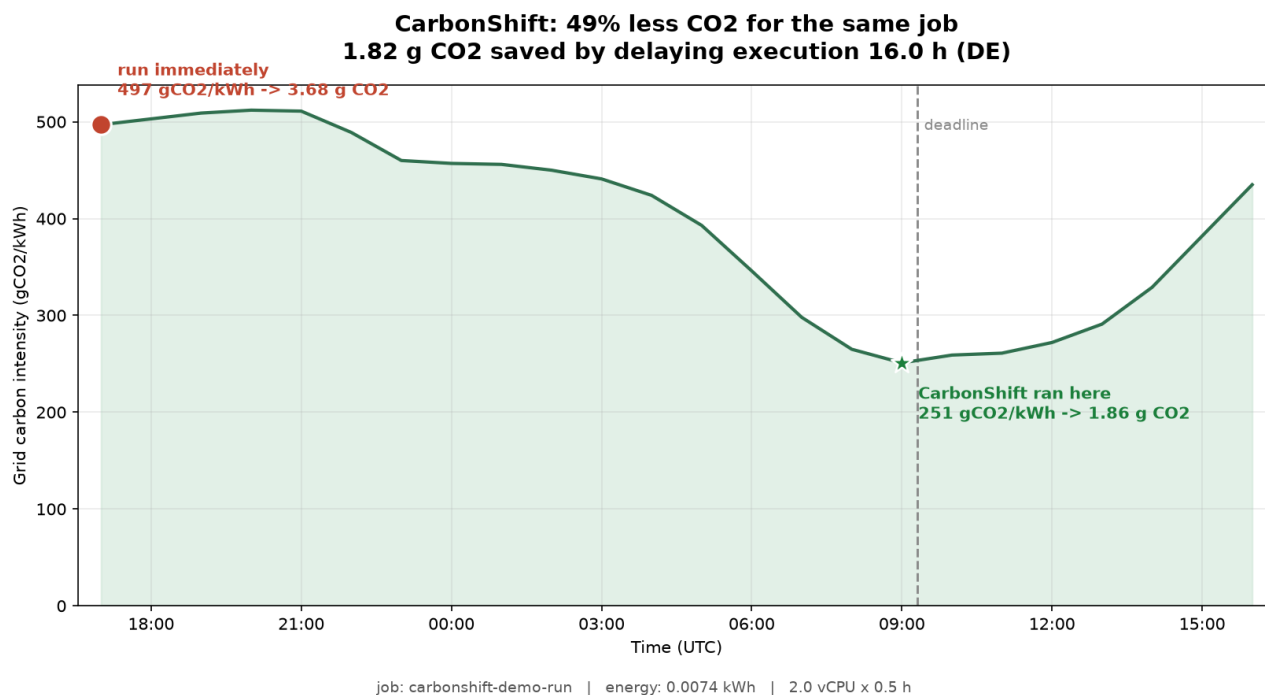


Figure 11.1 - one decision, rendered from its recorded run record by demo/chart.py.

11.3 Cost

The complete body of work, ten Fargate runs, image storage, log ingestion and every schedule, cost well under one US cent. Clusters, IAM roles and one-time schedules carry no standing charge.

12. Is this usable in production?

This section answers the question directly rather than listing caveats, because it is the question a practitioner will actually ask.

The short answer

Not yet, unattended, on work that matters. The decision engine and the cloud integration are real, tested and verified. What is missing is a recurring mode, failure handling, and a documented path to run your own workload. That is roughly three days of well-understood work on a foundation that already functions.

12.1 What is genuinely production quality

- **The decision logic.** 42 tests, including the cases that only live data revealed. The deadline is a hard exclusion, ties resolve deterministically, and it refuses to act when acting would make things worse.
- **The AWS integration.** EventBridge universal targets, two IAM roles scoped to least privilege, Fargate execution, self-deleting schedules. Verified firing within seconds of target, ten times.
- **Concurrency, in practice.** Ten independent schedules have coexisted without interference. The 'one job' limitation is about the tool not orchestrating a fleet, not about AWS being unable to.
- **The evidence trail.** Every decision recorded, every execution cross-checkable against CloudWatch, and a report that refuses to claim what it cannot prove.
- **Failure messages.** Every error names the missing permission, resource or command.

12.2 The three things that block real adoption

1. *It books one run. Real jobs recur.*

This is the most important gap. A nightly backup runs *every* night. CarbonShift books *one* execution at one clean hour and is then finished. There is no mode that says 'every night, at that night's cleanest hour'.

Using it on real work today would mean a daily cron that invokes CarbonShift, which then picks that day's best hour. That wrapper is small, but it does not exist, and without it nobody can adopt this for recurring work however good the rest is.

2. *No retry and no alerting*

If the Fargate task fails to start, nothing retries it and nobody is told. For a nightly backup that is serious: you would believe it had run.

The project's own doctor demonstrates the gap. It reports runs that were scheduled but produced no completion log. It is right to report them, but nothing *acted* on them, and nothing alerted anyone at the time.

3. *Running your own workload is undocumented*

The shipped worker processes a fixed dataset and prints a checksum. It exists to prove the mechanism. To shift a real job you replace it, either by editing the worker or by pointing the task definition at a container image you already have.

The mechanism supports both. `deploy.py --image-uri` has always accepted an arbitrary image. Neither path is written down, so a newcomer sees a working scheduler that runs a toy job and no obvious route from there to their own.

12.3 Smaller limitations, stated plainly

Limitation	Detail
The algorithm is simple	A filter and a minimum. The effort was in the integration, the error handling and the verification, not the computer science.
Absolute savings are small	10.33 g across ten runs. This is a proof of mechanism. The value is that it generalises, not that these runs mattered.
Energy is approximated	See section 2.3.1. The percentage is exact; the gram figures use a published coefficient because no cloud provider meters per-task power.
Forecast accuracy is assumed	A wrong forecast produces a wrong decision and nothing detects it.
Hourly granularity	A deadline shorter than an hour usually has no eligible slot, and the system says so rather than approximating.
Single cloud, single region	AWS only. The decision logic would carry over unchanged; the booking and provisioning would need rewriting.
No infrastructure as code	deploy.py is an idempotent script, not Terraform. Adequate at this scale, not at an organisation's.
Log retention is 14 days	The evidence underpinning a compliance report expires. Longer retention is a configuration change, but nobody is currently prompted to make it.

12.4 The criticism that would land hardest

If one objection is going to stick, it is this: **the saving is calculated against a counterfactual, not measured against an alternative that actually happened.**

CarbonShift proves the job ran at the chosen hour, and proves the forecast said that hour was cleaner. It does not re-query actual grid intensity after execution, and it never runs the job at the baseline hour to compare. The avoided emissions are modelled.

Closing that loop, by recording actual intensity at execution time and reporting realised rather than projected savings, is the clearest single improvement available and would materially strengthen every figure in section 11.

13. How another developer adopts it

13.1 There is no website

CarbonShift is not a hosted service. There is no sign-up and nowhere to paste credentials. It is a tool deployed into the operator's own AWS account, like Terraform or Prometheus.

Keys never leave the operator's machine. The `.env` file sits on their laptop. There is no server belonging to the project for anything to be sent to. The author cannot tell who is using it. That property is why developers can trust it, and it is why `.env` was excluded from git in the very first commit.

13.2 The adoption path

Stage	What is needed	Time
See the decision and the chart	Python 3.10+, an Electricity Maps token	~1 minute
Run the worker locally	+ Docker	~3 minutes
Schedule and execute real jobs	+ an AWS account, the AWS CLI configured, Docker running	~15 minutes

The first stage needs no cloud account at all. `python setup.py` checks for each prerequisite, says what each is for, and links to anything missing. It then offers to provision AWS and write every resulting setting into `.env` itself, so there is no copying of ARNs by hand.

13.3 Using it on Azure or GCP

Not supported today, but the split described in section 4.2 makes the position concrete.

What works on any cloud right now

The decision engine is cloud-agnostic. An Azure user can run the dry run, read the chosen timestamp out of the run record, and feed it into their own scheduling:

```
python -m src.scheduler --payload "my-job" --deadline-hours 12 --dry-run
python -c "import json; d=json.load(open('runs/latest.json')); \
    print(d['chosen']['timestamp'], d['co2_saved_percent'])"
```

CarbonShift answers *when*; Azure Container Apps Jobs, Logic Apps or Azure Automation handles *run it*.

What a real port would require

Piece	AWS today	Azure equivalent	Effort
Scheduling	EventBridge Scheduler	Logic Apps or a Container Apps Job schedule	~100 lines
Execution	ECS Fargate	Azure Container Apps Jobs or ACI	config only
Provisioning	infra/deploy.py (boto3)	azure-mgmt-* SDK	~300 lines
Decision logic	carbon_api.py, pick_best_slot	unchanged	zero

Two files, and the decision logic is untouched. That is a contained port rather than a redesign, and it is a direct consequence of not letting boto3 leak into the algorithm.

14. Appendix

14.1 Command reference

```
# Everything, as a menu with no flags to remember
python carbonshift.py

# Guided setup: prerequisites, region, API token, AWS, AI scanner
python setup.py

# Decide only, no AWS contact
python -m src.scheduler --payload "job" --deadline-hours 12 --dry-run

# Schedule for real
python -m src.scheduler --payload "job" --deadline-hours 12

# Bound the window at both ends, for work that cannot start early
python -m src.scheduler --payload "job" \
    --not-before 2026-09-17T12:00:00Z --deadline 2026-09-17T18:00:00Z

# Find which of your jobs could be shifted
python -m src.scanner --cron /etc/crontab
python -m src.scanner --aws --engine claude

# What has been saved, and whether each run really happened
python -m src.history --verify --chart

# A filable carbon reduction report
python -m src.report --month 2026-09 --verify --csv

# Live dashboard, works offline once open
python demo/dashboard.py --watch

# Diagnose a broken setup
python -m src.doctor

# Provision AWS (idempotent)
python infra/deploy.py

# Tests
python -m pytest tests/ -q
```

14.2 Repository

Item	Value
URL	github.com/RamyaSamudralaa/carbonshift
Licence	MIT
Commits	28, incremental
Tracked files	32
Python	~7,290 lines, 3.10+
Tests	177, all passing
Secrets in history	None. Verified by scanning every commit.
Dependencies	requests, boto3, matplotlib, python-dotenv, pytest

14.3 Teardown

Everything created for this project, and how to remove it:

```
aws ecs deregister-task-definition --task-definition carbonshift-worker:1
aws ecs delete-cluster --cluster carbonshift-cluster
aws ecr delete-repository --repository-name carbonshift-worker --force
aws logs delete-log-group --log-group-name /ecs/carbonshift-worker
aws iam delete-role-policy --role-name carbonshift-scheduler-role \
--policy-name carbonshift-scheduler-runtask
aws iam delete-role --role-name carbonshift-scheduler-role
aws iam detach-role-policy --role-name carbonshift-ecs-execution-role \
--policy-arn arn:aws:iam::aws:policy/service-role/AmazonECSTaskExecutionRolePolicy
aws iam delete-role --role-name carbonshift-ecs-execution-role
```

14.4 How this project was built

CarbonShift was built by the author working with an AI coding assistant (Claude), in a continuous paired session over eight days.

The author set the specification and held its scope, chose the cloud provider and grid zone, owned the AWS account and every credential in it, and made each decision the project turned on: to add the refusal guard described in section 9.4, to prioritise usability over further features, to build the compliance report, and to require that the setup tooling serve someone with no assistant beside them. Several of the defects in section 9 exist as fixed rather than shipped because the author pressed on gaps the assistant had not surfaced, including the Docker reporting fault in 9.8.

The assistant wrote the implementation, diagnosed the defects, produced the test suite, and drafted this report.

This is recorded because a technical report should be accurate about its own provenance, and because the division of labour is itself the honest answer to how a single undergraduate produced a verified cloud system in eight days.

14.5 Provenance of the figures in this report

- Carbon intensity values: Electricity Maps, retrieved live at the time of each decision.
- Execution times and exit codes: AWS CloudWatch Logs and the ECS API.
- Test counts: pytest, run immediately before this report was generated.
- Repository statistics: git, at commit 28.
- Energy coefficient: 0.0074 kWh per vCPU-hour, a published approximation, applied identically to baseline and shifted cases so that it cancels from every percentage.